

<HLS Programming with FPGAs>

AWS F1 Hello World

Choi, Young-kyu

Acknowledgement

- Many of the figures and contents in this part are from
 - <https://github.com/aws/aws-fpga>
 - Announcing Amazon EC2 F1 Instances with Custom FPGA (<https://youtu.be/5sDVprory3c>)
 - Amazon EC2 F1 SDAccel Developer Lab (<https://youtu.be/RvTSyVa6bCw>)
 - Amazon EC2 Foundation (AWS re: Invent)
 - <https://aws.amazon.com/ec2>

Learning Objectives

- Follow the hello world project for EC2 F1:
 - <https://github.com/aws/aws-fpga/blob/master/Vitis/README.md>
- 1. Build the host application, and the Xilinx FPGA binary
- 2. Create a bucket and Amazon FPGA Image (AFI)
- 3. Run the FPGA accelerated application on AWS FPGA instances

Start m5.xlarge instance



FPGA Developer AMI

★★★★★ (7) | 1.11.0 [Previous versions](#) | By [Amazon Web Services](#)

Linux/Unix, CentOS 7.5 | 64-bit (x86) Amazon Machine Image (AMI) |

Updated: 7/21/21

Select

Free tier eligible

☐	m5	m5.large	2	8
☒	m5	m5.xlarge	4	16
☐	m5	m5.2xlarge	8	32



Instances (1/1) [Info](#)

Instance state: running X

Clear filters

☒	Name	Instance ID	Instance state	Instance type
☒	-	[REDACTED]	Stopped	m5.xlarge

Launch instances

Launch instance from temp

Connect

Stop instance

Start instance

Reboot instance

- Github and Environment Setup

```
git clone https://github.com/aws/aws-fpga.git $AWS_FPGA_REPO_DIR  
cd $AWS_FPGA_REPO_DIR  
source vitis_setup.sh
```

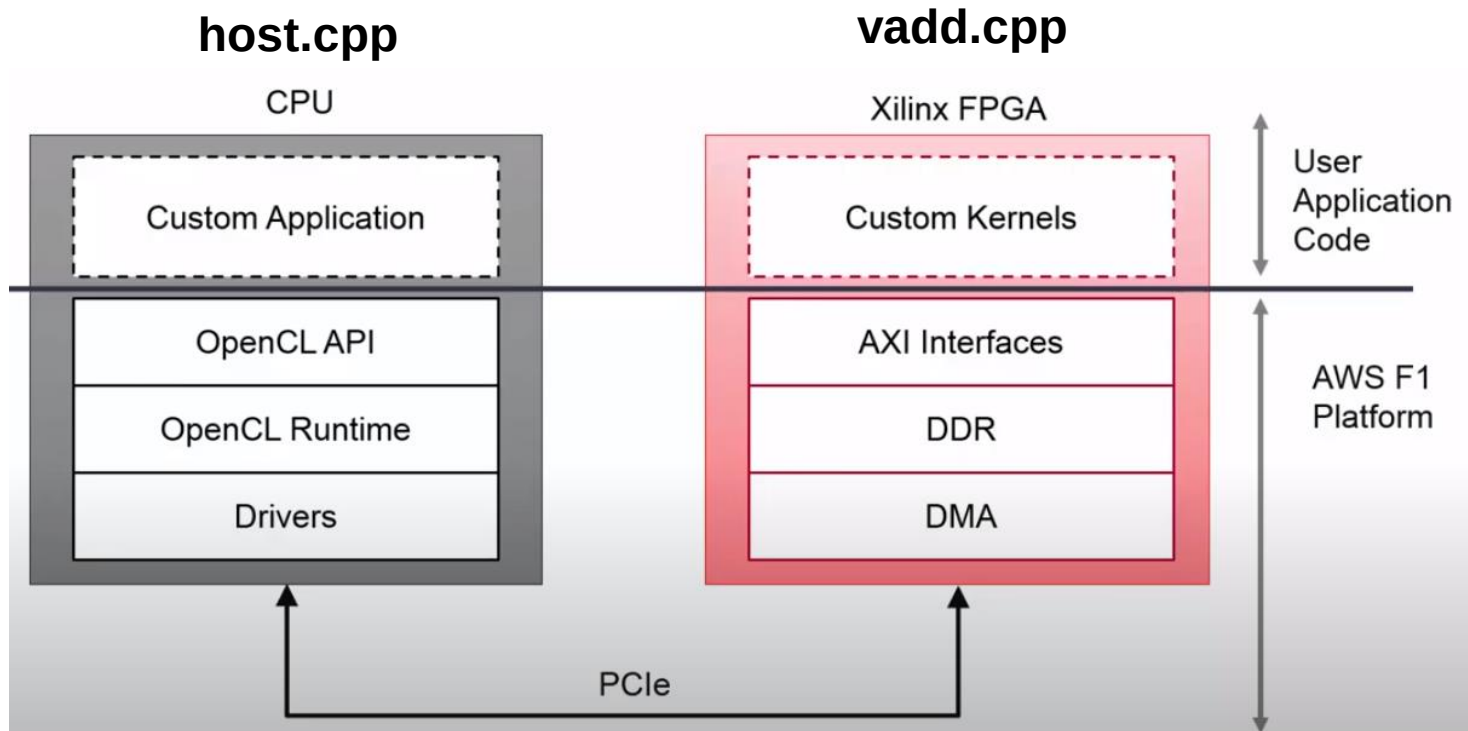
- Build the Host Application and Xilinx FPGA Binary

```
cd $VITIS_DIR/examples/xilinx/hello_world  
make clean  
make TARGET=hw DEVICE=$AWS_PLATFORM all
```

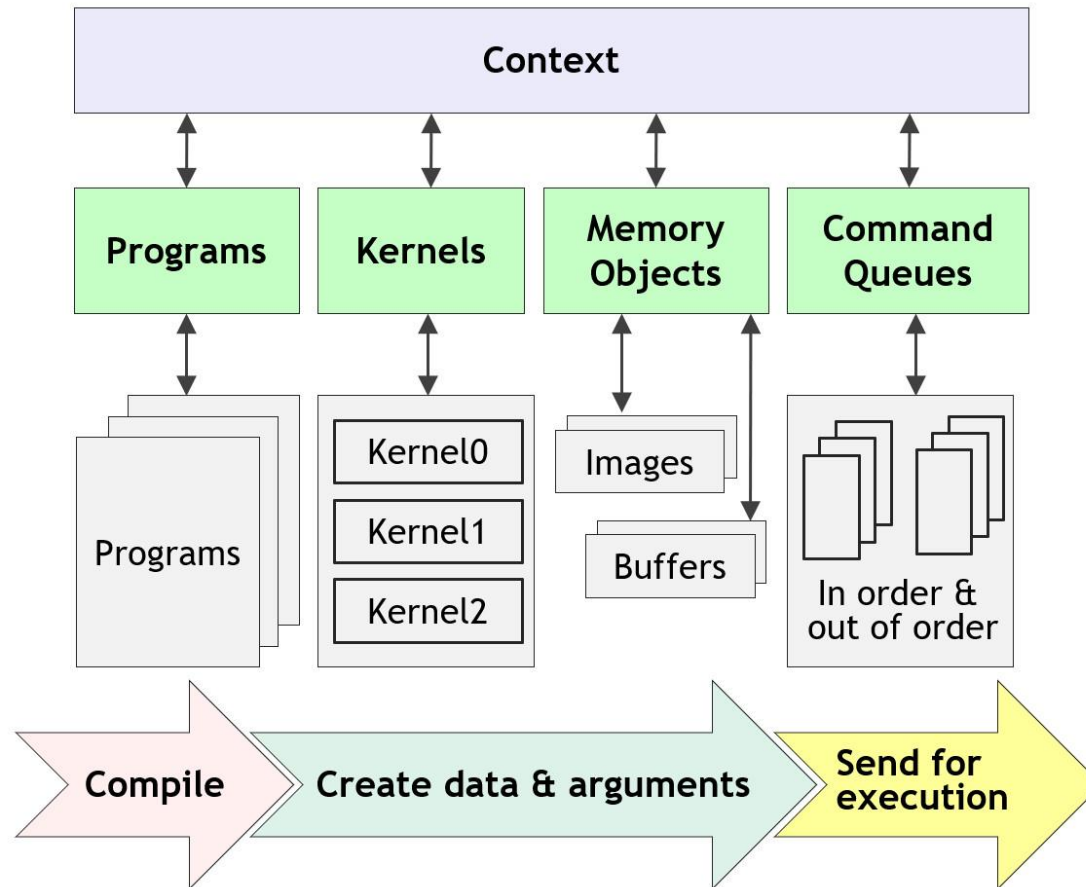
Hello World Source Code

```
$ cd $VITIS_DIR/examples/xilinx/hello_world/src  
$ ls
```

<OpenCL computation model>



Executing OpenCL Program



<https://github.com/KhronosGroup/OpenCL-Guide>

Host Code (src/host.cpp)

- Written in OpenCL
- Content of host.cpp:
 1. Look for an FPGA device

```
auto devices = xcl::get_xil_devices();
```

2. Create context, command queue, and program

```
OCL_CHECK(err, context = cl::Context(device, NULL, NULL, NULL, &err));  
OCL_CHECK(err, q = cl::CommandQueue(context, device,  
                                     CL_QUEUE_PROFILING_ENABLE, &err));  
cl::Program program(context, {device}, bins, NULL, &err);
```

3. Allocate buffers in device global memory

```
OCL_CHECK(err, cl::Buffer buffer_in1(  
    context, CL_MEM_USE_HOST_PTR | CL_MEM_READ_ONLY,  
    vector_size_bytes, source_in1.data(), &err));
```

(.. and for buffer_in2, buffer_output as well)

- Content of host.cpp (continued):

4. Set kernel arguments

```
OCL_CHECK(err, err = krnl_vector_add.setArg(0, buffer_in1));  
OCL_CHECK(err, err = krnl_vector_add.setArg(1, buffer_in2));  
OCL_CHECK(err, err = krnl_vector_add.setArg(2, buffer_output));  
OCL_CHECK(err, err = krnl_vector_add.setArg(3, size));
```

5. Copy input data into FPGA device memory

```
OCL_CHECK(err, err = q.enqueueMigrateMemObjects({buffer_in1, buffer_in2},  
                                                0 /* 0 means from host*/));
```

6. Launch kernel

```
OCL_CHECK(err, err = q.enqueueTask(krnl_vector_add));
```

7. Copy output data from FPGA device memory

```
OCL_CHECK(err, err = q.enqueueMigrateMemObjects({buffer_output},  
                                                CL_MIGRATE_MEM_OBJECT_HOST));  
q.finish();
```

Kernel Code (src/vadd.cpp)

- Written in C++
- Has HLS-specific compiler directives
- Composed of 4 functions (top function + 4 function calls)

Top function:

```
extern "C" {  
  
void vadd(hls::vector<unsigned int, 16>* in1,  
          hls::vector<unsigned int, 16>* in2,  
          hls::vector<unsigned int, 16>* out,  
          int size) {  
    #pragma HLS INTERFACE m_axi port = in1 bundle = gmem0  
    #pragma HLS INTERFACE m_axi port = in2 bundle = gmem1  
    #pragma HLS INTERFACE m_axi port = out bundle = gmem0  
  
    static hls::stream<hls::vector<unsigned int, 16> > in1_stream("input_stream_1");  
    static hls::stream<hls::vector<unsigned int, 16> > in2_stream("input_stream_2");  
    static hls::stream<hls::vector<unsigned int, 16> > out_stream("output_stream");  
  
    assert(size % 16 == 0);  
    int vSize = size / 16;  
    #pragma HLS dataflow  
  
    load_input(in1, in1_stream, vSize);  
    load_input(in2, in2_stream, vSize);  
    compute_add(in1_stream, in2_stream, out_stream, vSize);  
    store_result(out, out_stream, vSize);  
}  
}
```

Definition of the functions being called:

```
static void load_input(hls::vector<unsigned int, 16>* in,
                      hls::stream<hls::vector<unsigned int, 16> >& inStream,
                      int vSize) {
mem_rd:
    for (int i = 0; i < vSize; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size
        inStream << in[i];
    }
}

static void compute_add(hls::stream<hls::vector<unsigned int, 16> >& in1_stream,
                       hls::stream<hls::vector<unsigned int, 16> >& in2_stream,
                       hls::stream<hls::vector<unsigned int, 16> >& out_stream,
                       int vSize) {
execute:
    for (int i = 0; i < vSize; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size
        out_stream << (in1_stream.read() + in2_stream.read());
    }
}

static void store_result(hls::vector<unsigned int, 16>* out,
                        hls::stream<hls::vector<unsigned int, 16> >& out_stream,
                        int vSize) {
mem_wr:
    for (int i = 0; i < vSize; i++) {
#pragma HLS LOOP_TRIPCOUNT min = c_size max = c_size
        out[i] = out_stream.read();
    }
}
```

- Quiz: What computation is this program doing?

AWS Cloud Storage Service

The screenshot shows the AWS Management Console's 'Services' page. The left sidebar contains 'Favorites' (Resource Groups & Tag Editor) and 'Recently visited' (EC2, Console Home, Billing, Support). The main area is titled 'All services' and lists various AWS services categorized by icon. The 'Storage' category is expanded, showing 'S3', 'EFS', and 'FSx'. The 'S3' service is highlighted with a red rectangular box.

Services ▲

Search for services, features

★ Favorites

Resource Groups & Tag Editor

Recently visited

- EC2
- Console Home
- Billing
- Support

All services

Compute

- EC2
- Lightsail ↗
- Lambda
- Batch
- Elastic Beanstalk
- Serverless Application Repository
- AWS Outposts
- EC2 Image Builder
- AWS App Runner

Containers

- Elastic Container Registry
- Elastic Container Service
- Elastic Kubernetes Service
- Red Hat OpenShift Service on AWS

Storage

- S3**
- EFS
- FSx

Create S3 Bucket

Buckets (0) [Info](#)

Buckets are containers for data stored in S3. [Learn more](#) 

  **Copy ARN** **Empty** **Delete** **Create bucket**

Create bucket [Info](#)

Buckets are containers for data stored in S3. [Learn more](#) 

General configuration

Bucket name

Bucket name must be unique and must not contain spaces or uppercase letters. [See rules for bucket naming](#) 

AWS Region

Create Folder

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you



Copy S3 URI

Copy URL

 Download

Open 

Delete

Actions ▼

Create folder

 Find objects by prefix

Folder

Folder name

dcp

Folder names can't contain

Folder

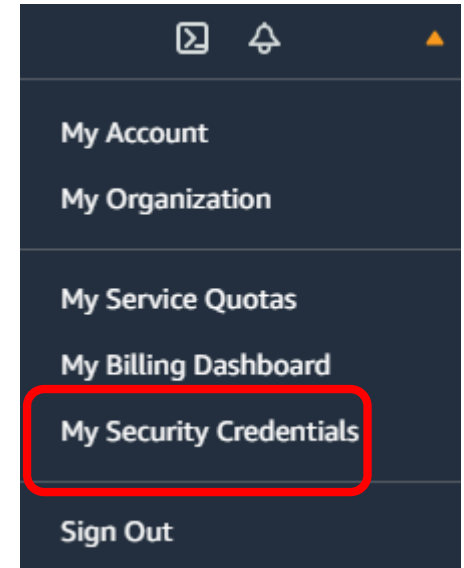
Folder name

logs

Folder names can't contain

Access Key ID

- My Security Credentials -> Access Keys



▲ Password

▲ Multi-factor authentication (MFA)

▼ Access keys (access key ID and secret access key)

Use access keys to make programmatic calls to AWS from the AWS CLI, Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time.

For your protection, you should never share your secret keys with anyone. As a best practice, we recommend frequent key rotation.

If you lose or forget your secret key, you cannot retrieve it. Instead, create a new access key and make the old key inactive. [Learn more](#)

Created	Access Key ID	Last Used	Last Used Region	Last Used Service	Status	Actions
---------	---------------	-----------	------------------	-------------------	--------	---------

Create New Access Key

Access Key ID

Create Access Key

✔ Your access key (access key ID and secret access key) has been created successfully.

Download your key file now, which contains your new access key ID and secret access key. If you do not download the key file now, you will not be able to retrieve your secret access key again.

To help protect your security, store your secret access key securely and do not share it.

▼ Hide Access Key

Access Key ID:

Secret Access Key:

Download Key File

Close

Create S3 Bucket

\$ aws configure

```
[centos@ip-10.0.0.100 ~]$ aws configure
AWS Access Key ID [None]: <from My Security Credentials>
AWS Secret Access Key [None]: 
Default region name [None]: us-east-1
Default output format [None]: json
```

Test:

```
[centos@ip-10.0.0.100 ~]$ aws s3 ls <your bucketname>
PRE dcp/
PRE logs/
```

Please wait for bitstream generation to finish... 😊

Creating an AWS Image

- When compilation is finished, search for .xclbin file:
 - `find . -name "*.xclbin"`

```
[centos@ip-10.0.0.1 ~]$ ls ./build_dir.hw.xilinx_aws-vu9p-f1_shell-v04261818_201920_2/  
link      vadd.link.ltx      vadd.link.xclbin.link_summary  v++_link_vadd.link_guidance.json  
logs      vadd.link.xclbin    vadd.xclbin                    v++_link_vadd.link_guidance.pb  
reports   vadd.link.xclbin.info  vadd.xclbin.package summary
```

- Transform .xclbin file to .awsxclbin file:

`$VITIS_DIR/tools/create_vitis_afi.sh`

`-xclbin=./build_dir.hw.xilinx_aws-vu9p-f1_shell-v04261818_201920_2/vadd.xclbin -o=vadd`
`-s3_bucket=<your-bucket-name>`
`-s3_dcp_key=dcp -s3_logs_key=logs`

Tracking Status of Your AFI

- Find for *_afi_id.txt
- Look into the file for FpgaImageID

```
{  
  "FpgaImageId": "afi-<your AFI ID here>  
  "FpgaImageGlobalId": "agfi-  
}
```

- Type:
 - **\$ aws ec2 describe-fpga-images --fpga-image-ids afi-<AFI ID>**
- Wait until you see:

```
...  
  "State": {  
    "Code": "available"  
  },  
  ...
```

Copy Files to Bucket

- **aws s3 cp vadd.awsxcclbin s3://<your_bucketname>**
- **aws s3 cp hello_world s3://<your_bucketname>**

Create F1 Instance

Step 2: Choose an Instance Type

	user1	user1.f2xlarge	40	192	
☒	f1	f1.2xlarge	8	122	
☐	f1	f1.4xlarge	16	244	
☐	f1	f1.16xlarge	64	976	

Execute AFI on F1

- Github and Environment Setup

```
git clone https://github.com/aws/aws-fpga.git $AWS_FPGA_REPO_DIR
cd $AWS_FPGA_REPO_DIR
source vitis_setup.sh
source vitis_runtime_setup.sh
cd $VITIS_DIR/examples/xilinx/hello_world
```

- Copy files from bucket

```
aws s3 cp s3://<your_bucketname>/vadd.awsxcclbin .
aws s3 cp s3://<your_bucketname>/hello_world .
```

- Execute your AFI

```
chmod +x hello_world
./hello_world vadd.awsxcclbin
```

Final Output Screen

```
[centos@ip-          hello_world]$ ./hello_world vadd.awsxclbin
Found Platform
Platform Name: Xilinx
INFO: Reading vadd.awsxclbin
Loading: 'vadd.awsxclbin'
Trying to program device[0]: xilinx_aws-vu9p-fl_shell-v04261818_201920_2
Device[0]: program successful!
TEST PASSED
```

- Additional exercise:
 1. Modify host.cpp file so that it will display execution time in seconds
 2. Modify host.cpp and vadd.cpp file so that it will do vector add of double precision floating-point numbers, rather than integer numbers

General Advices to Avoid Overbilling

- Stop your instance when not being used
- Delete your bucket content regularly, especially dcp/ and logs/ folders
- Minimize your F1 usage if possible, and use cheaper nodes for development
- Do not over-create instances you do not need (SSD incurs cost even if not used)
- Consider termination of instances if not going to be used for a long time

Summary

- `host.cpp` is executed on host CPU, and `vadd.cpp` is executed on FPGA
- Hello world is a vector add application
- It takes several hours to create an FPGA bitstream
- Used bucket to transfer AFI to F1, and executed the FPGA image